

**FACULTY OF NATURAL SCIENCE
DEPARTMENT OF COMPUTER SCIENCE**

CSE 2101: Software Engineering

**System Design Document: Student Records Management System
(SRMS) Mobile Application**



Client: Software Services & Educational Technology Applications
(SSETA)

Date: December 12, 2025

Group 5

Kaylee Carter (1051799)

Khaleel Khan (1054193)

Ravindra Persaud (1052507)

Vimla Persaud (1052339)

Tulsidai Singh (1052682)

Table of Contents

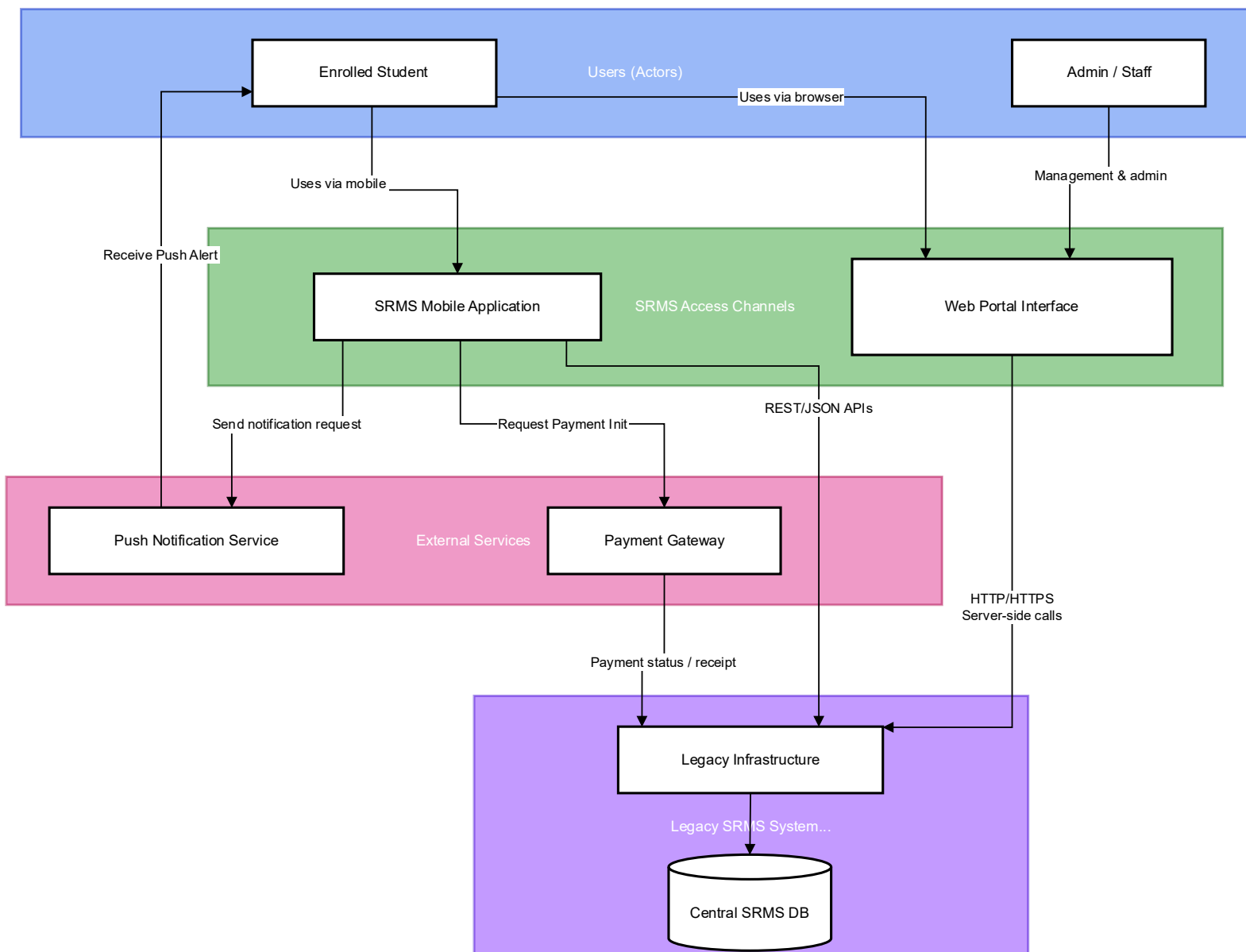
1.0	Introduction.....	3
2.0	Context Model	3
2.1	Context Diagram Figure	3
2.2	Diagram Description.....	4
3.0	Use Case Model	5
3.1	Use Case Diagram Figure	5
3.2	Use Case Diagram Description.....	6
3.3	Use Case Specifications (UC-01 to UC-05)	7
4.0	Sequence Diagrams.....	11
4.1	Complex Scenario.....	11
4.2	Simple Scenario	12
5.0	Detailed Class Diagram	13
5.1	Detailed Class Diagram Figure.....	13
5.2	Detailed Class Diagram Description.....	14
6.0	Activity Diagram	15
6.1	Activity Diagram Figure.....	15
6.2	Activity Diagram Description.....	16
7.0	System Architectural Design.....	17
7.1	System Architectural Diagram Figure	17
7.2	System Architectural Diagram Description	18
7.3	Scenario: Course Registration.....	19
	Collaborative Summary	20
	Division of Labour:.....	20
	Collaboration Strategies and General Experiences:.....	20
	Challenges and Resolutions:	20
	Appendix A: High-Resolution Diagram Links	21

1.0 Introduction

This System Design document details the architectural and logical structure of the Student Records Management System (SRMS) Mobile Application. It builds on the requirements defined in the SRS by turning the functional requirements into technical models to meet the University of Guyana's needs.

2.0 Context Model

2.1 Context Diagram Figure

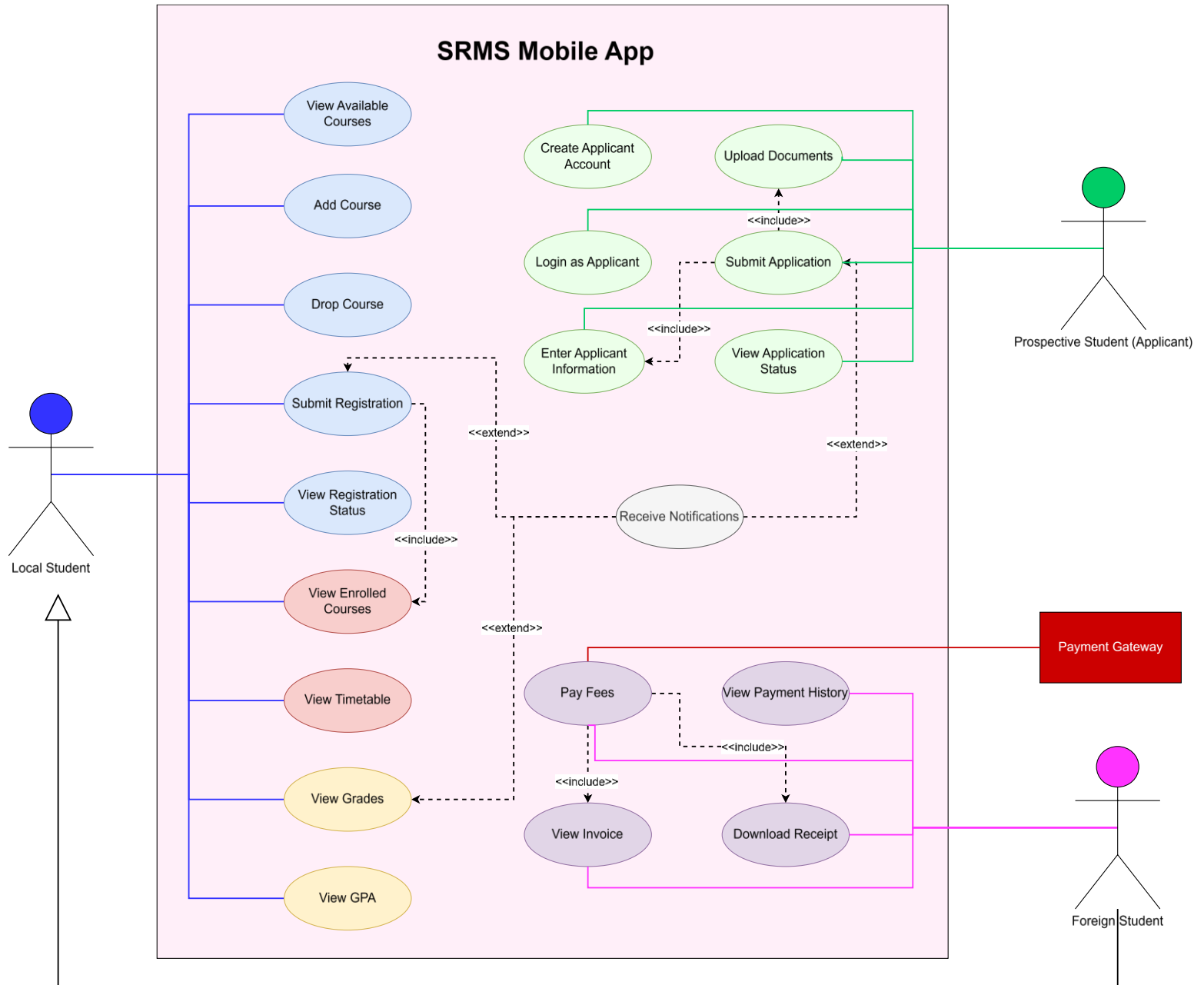


2.2 Diagram Description

The context model shows how the SRMS mobile application and the existing SRMS web portal operate at the same time as two access channels to the same system. Students can use either the mobile app or the web/mobile webpage, while administrators continue to manage records through the web portal. Both channels connect to the legacy SRMS backend and central database, ensuring that all data remains consistent. The mobile application also integrates with external services, including a payment gateway for fee transactions and a push notification service for real-time alerts, all of which function within the existing SRMS environment.

3.0 Use Case Model

3.1 Use Case Diagram Figure



3.2 Use Case Diagram Description

The use case diagram above shows how different types of students (actors) will interact with the SRMS mobile app. The local student actor shown in blue can perform several actions grouped into 3 feature areas. The light blue use cases represent the registration process where a local student can view available courses, add courses, drop courses, submit registration and check their registration status. The light red use cases represent course schedule features where the student can view their enrolled courses and view their timetable. The yellow use cases represent academic features which include viewing grades and checking GPA. The prospective student actor is shown in green and connects to the application process. This includes creating an applicant account, logging in, entering personal information, uploading documents, submitting the application and viewing application status. Some actions have required steps so we used include relationships. For example submitting an application includes entering applicant information and uploading documents. Submitting registration includes viewing enrolled courses. There is also a separate use case called Receive Notifications (in grey). It doesn't belong to an actor, it extends other use cases such as submitting an application, submitting registration and viewing grades. This means notifications only happen after those actions. The foreign student actor inherits all actions from the local student because they follow the same academic and registration steps. However, foreign students also perform payment activities (shown in purple). These include pay fees, viewing invoices, viewing payment history and downloading receipts. The pay fees use case includes viewing invoices and downloading receipts because those steps are part of the payment process. Foreign student interact with another actor called Payment Gateway (shown in red) which handles payment processing. The diagram also uses colour coded lines so each actor's interaction match their actor colour.

3.3 Use Case Specifications (UC-01 to UC-05)

Use Case: UC-01 - Register for Courses

Field	Description
Actors	Student, SRMS API
Description	This use case describes the process where a student selects, validates, and submits their course registration for the current semester.
Preconditions	1. The student is logged into the app. 2. The registration period is open 3. The student has no holds preventing registration.
Main Flow	1. The Student opens the Registration screen. 2. The System retrieves available courses, prerequisites and the student's current status. 3. The Student adds desired courses to their provisional list. 4. The System validates the selection against prerequisites and programme rules. 5. The Student navigates to the “Preview” screen. 6. The System displays the selected courses and total credits. 7. The Student confirms and taps “Submit.” 8. The System sends the data to the SRMS API. 9. The System displays a success message and the updated registration status (e.g. “Submitted”).
Postconditions	The student's registration is recorded in the central SRMS database, and the status is updated to “Submitted” pending approval.
Exceptions	4a. Validation Failed: The system detects a missing prerequisite. It displays an error message and prevents the course from being added. 8a. Submission Failed: The API is unreachable. The system displays a “Connection Error” message and saves the draft locally.

Use Case UC-02 - Apply to University

Field	Description
Actors	Prospective Student, SRMS API
Description	This use case allows a new user to complete and submit an application for a university programme, including document uploads.
Preconditions	The user has created an applicant account and is logged in.
Main Flow	1. The Student selects “Apply to UG.” 2. The Student completes the Personal Information and Address forms. 3. The System validates and saves the data. 4. The Student enters Education and Employment history. 5. The Student selects a Programme from the available list. 6. The Student uploads required support documents (IDs, Certificates). 7. The Student reviews the full application summary. 8. The Student confirms submission. 9. The System transmits data to the backend and displays a confirmation message.
Postconditions	The application is created in the SRMS database with a status of Submitted or Under Review.
Exceptions	6a. Upload Error: The file format is unsupported or too large. The system displays an error and prompts for a valid file. 9a. Duplicate Application: The system detects an existing application for this user. It displays a notification and redirects the user to the status tracking screen.

Use Case: UC-03 - View Timetable

Field	Description
Actors	Student, SRMS API
Description	This use case allows a registered student to view their class schedule for the current semester.
Preconditions	1. The student is logged in. 2. The student is officially enrolled in at least one course.
Main Flow	1. The Student selects the “Timetable” option. 2. The System retrieves the student's enrolled courses and associated time/venue data. 3. The System displays the schedule in a weekly format. 4. The Student toggles between weekly and daily views. 5. The Student taps a specific class for details.
Postconditions	The student has viewed their schedule. No data is changed.
Exceptions	2a. No Classes: If the student is not registered, the system displays “No registered courses found.” 3a. Hybrid Class: If a class is marked “Hybrid,” the system displays a note advising the student to contact the lecturer.

Use Case: UC-04 - View Academic Record

Field	Description
Actors	Student, SRMS API
Description	This use case allows a student to view their historical grades, current semester results, and cumulative Grade Point Average (GPA).
Preconditions	1. The student is logged into the application. 2. Grades have been published by the Registry.
Main Flow	1. The Student selects “My Grades” from the home dashboard. 2. The System requests the student’s academic history from the SRMS database. 3. The System calculates and displays the current cumulative GPA. 4. The System lists all completed semesters. 5. The Student taps on a specific semester. 6. The System expands the view to show courses taken in that semester and the final letter grade received for each.
Postconditions	The student has successfully viewed their academic performance. No data is modified.
Exceptions	2a. Financial Hold: If the student has a financial hold that blocks grade viewing, the system displays a notification: “Grades are unavailable due to an outstanding balance. Please contact the Bursary.”

Use Case: UC-05 - Make Tuition Payment

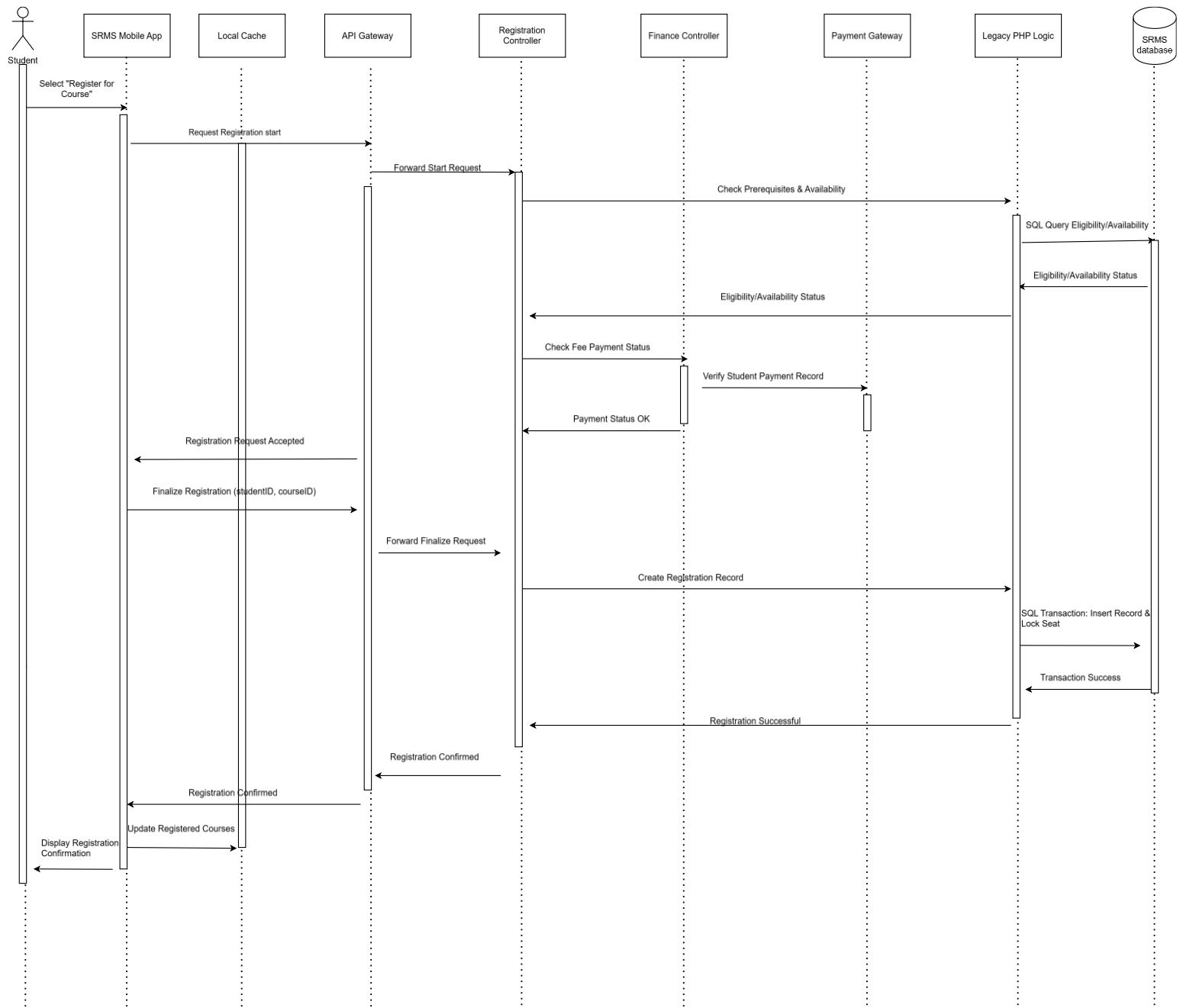
Field	Description
Actors	Student (Foreign/CARICOM), MMG System, Registry Staff
Description	This use case describes how a non-resident student pays tuition fees using MMG or manual receipt upload to regain access to grades.
Preconditions	1. The user is logged into the app. 2. The user is marked as Foreign or CARICOM 3. An invoice with an outstanding balance exists.
Main Flow	A. MMG Payment (Automatic) 1. Student selects “Pay with MMG.” 2. System redirects to MMG interface. 3. Student completes transaction. 4. System receives success signal. 5. System updates status to Cleared and unlocks grades B. Manual Payment (Upload) 1. Student selects “Upload Receipt.” 2. Student uploads image of bank receipt. 3. System saves image and sets status to Pending Verification. 4. Registry Staff (Backend) verifies receipt. 5. System sends “Payment Approved” notification.
Postconditions	The student's balance is updated. If paid via MMG, academic blocks are removed immediately.
Exceptions	A1. MMG Failed: Transaction fails. System displays error and retains blocks. B1. Receipt Rejected: Staff rejects the uploaded image (e.g., blurry). System sends “Resubmit Receipt” notification.

4.0 Sequence Diagrams

4.1 Complex Scenario

Description

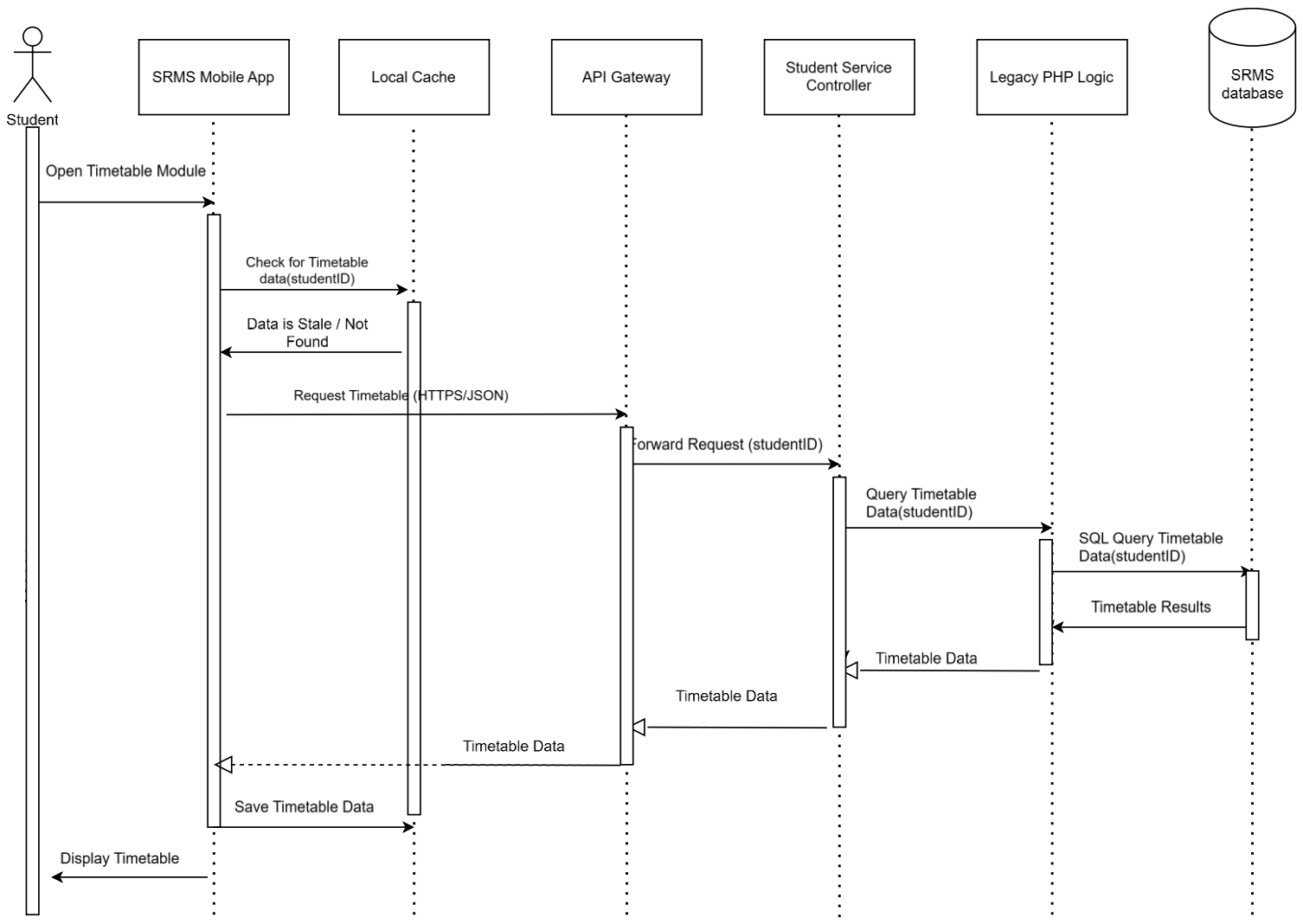
The Course Registration Diagram (Complex): This shows a much more complex job which is signing up for a course. Since this task changes important records, it's a long process. The Registration Controller first checks two major things: are you eligible (prerequisites) and have you paid your fees (by talking to the Finance Controller and Payment Gateway)? Only once all these checks pass does the system allow the Legacy PHP Logic to officially save / commit the registration into the Central SRMS DB. This is a transactional process that makes sure the record is updated correctly and securely.



4.2 Simple Scenario

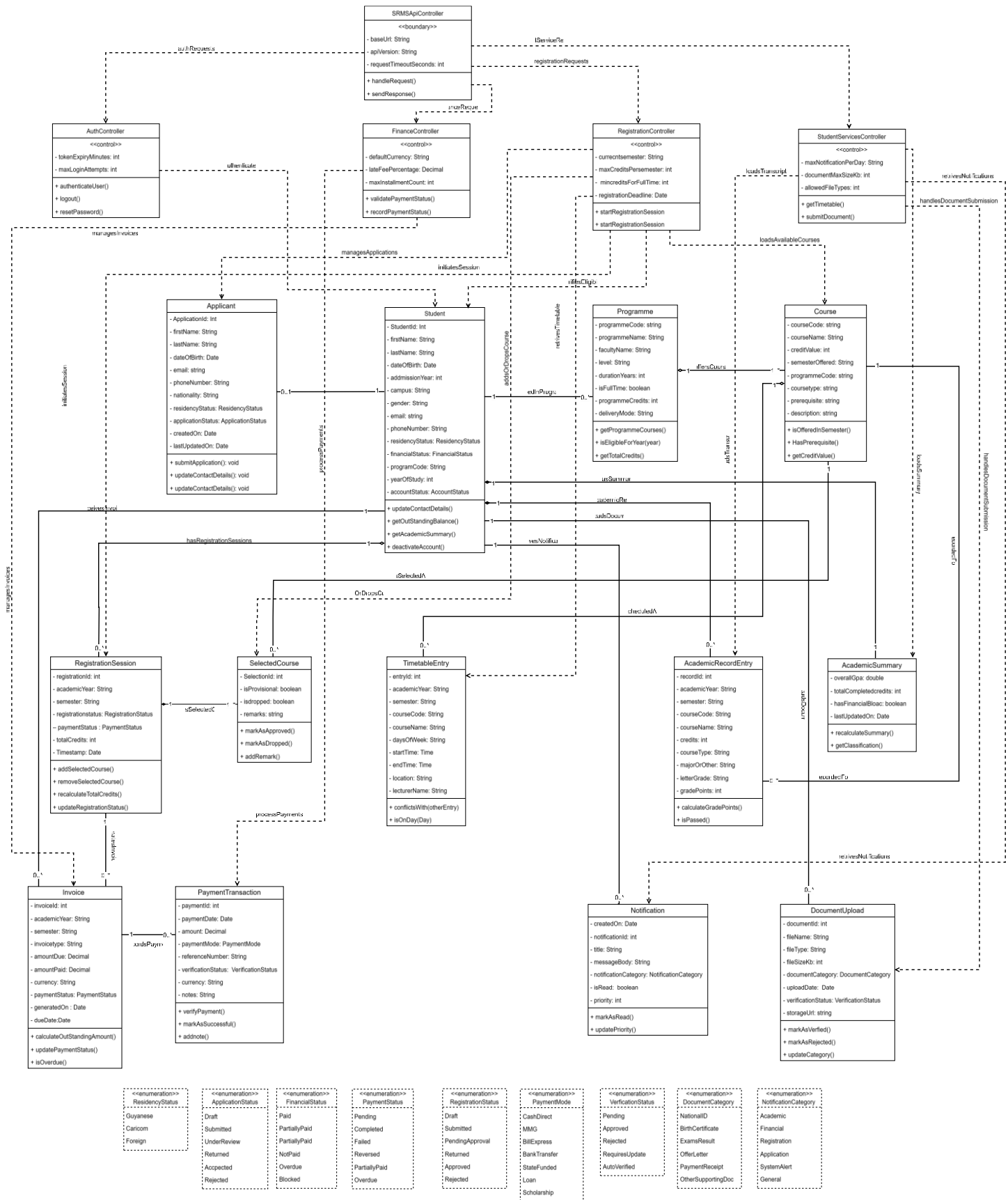
Description

The Timetable Diagram (Simple): This shows a quick and easy job which is just looking up your class schedule. The app first checks your phone's Local Cache to see if it already has the schedule saved. If not, it sends a request through the API Gateway to the Student Services Controller, which has to talk to the old Legacy PHP Logic to pull the information from the main Central SRMS DB. This is a read-only process, meaning it just gets data without changing anything.



5.0 Detailed Class Diagram

5.1 Detailed Class Diagram Figure



5.2 Detailed Class Diagram Description

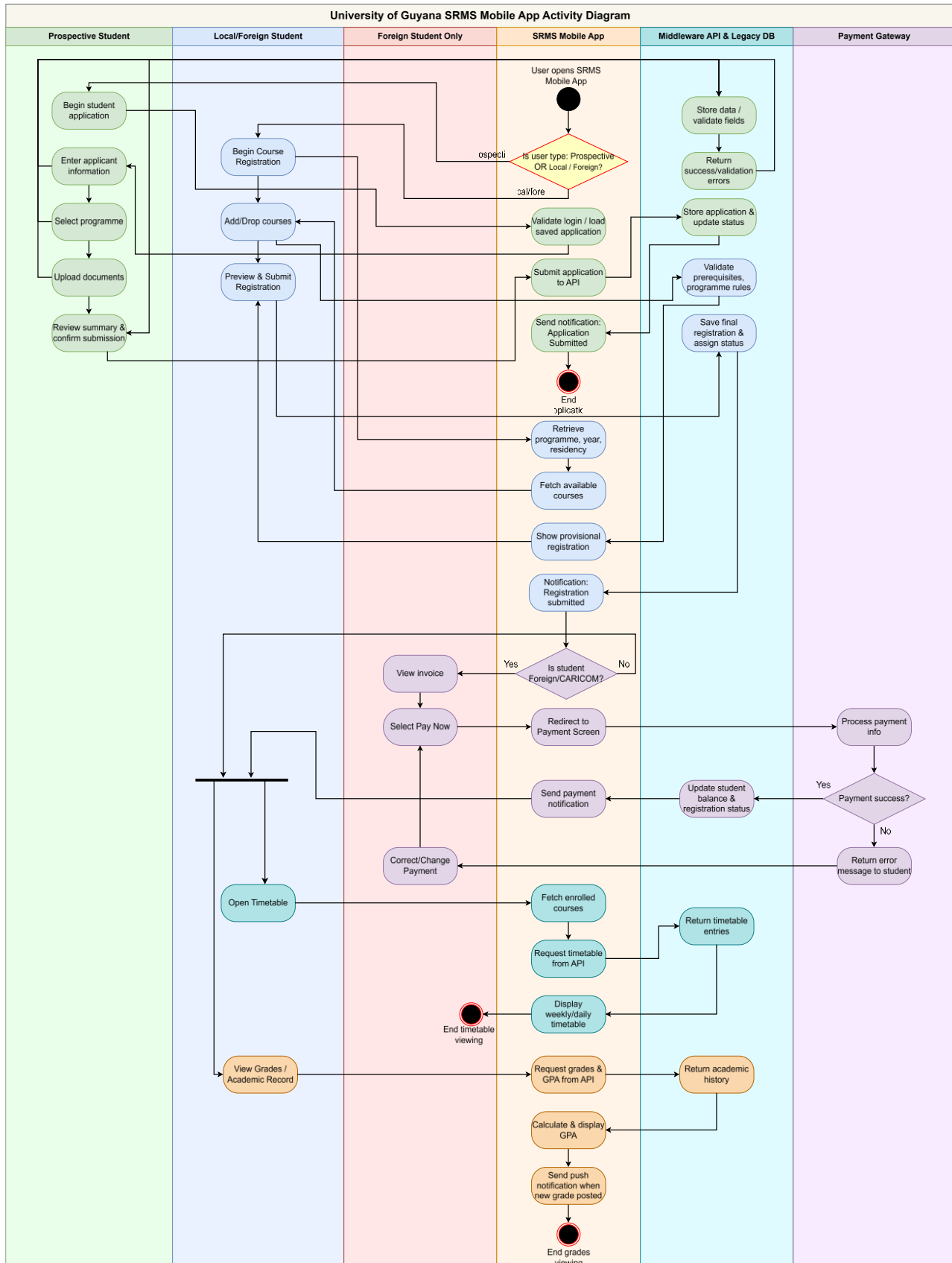
The class diagram for the SRMS mobile application using standard UML notation portrays a detailed structural view. The diagram identifies the essential domain classes such as Student, programme, courses, registration sessions, invoices and document upload, including their attributes, operations, and enumerations. The controller classes used within the diagram such as auth controller, registration controller, finance controller, and student service controller, represents the system's core logic and it facilitates the interaction between the user interface and the core mode. These controller classes would interact with the unified API allowing for both the web and mobile application to be synchronized.

Additionally, the association, multiplicities and relationship names are used within the diagram to show how classes interact with each other. For example, a student within the SRMS mobile application may have many registration sessions, notifications and invoices or payments and each has one academic summary. These relationships are shown using UML class diagram symbols, this allows them to provide an accurate dataflow and constraints within the system. It also utilizes enumerations, which are presented separately for clarity, meaning that there is no relationship lines between the classes and the enumeration. This was done to ensure that the diagram isn't too congested with lines, so can you easily follow the essential relationships within the diagram. By including them it allowed for the value sets being represented without crowding the main model.

Finally, the model provides a clear and accurate representation of the system's structure, showing all the necessary entities/ classes, controller logic, and any other supporting components that will support students to register, manage academics, financial processing and upload documents all within the SRMS mobile application.

6.0 Activity Diagram

6.1 Activity Diagram Figure



6.2 Activity Diagram Description

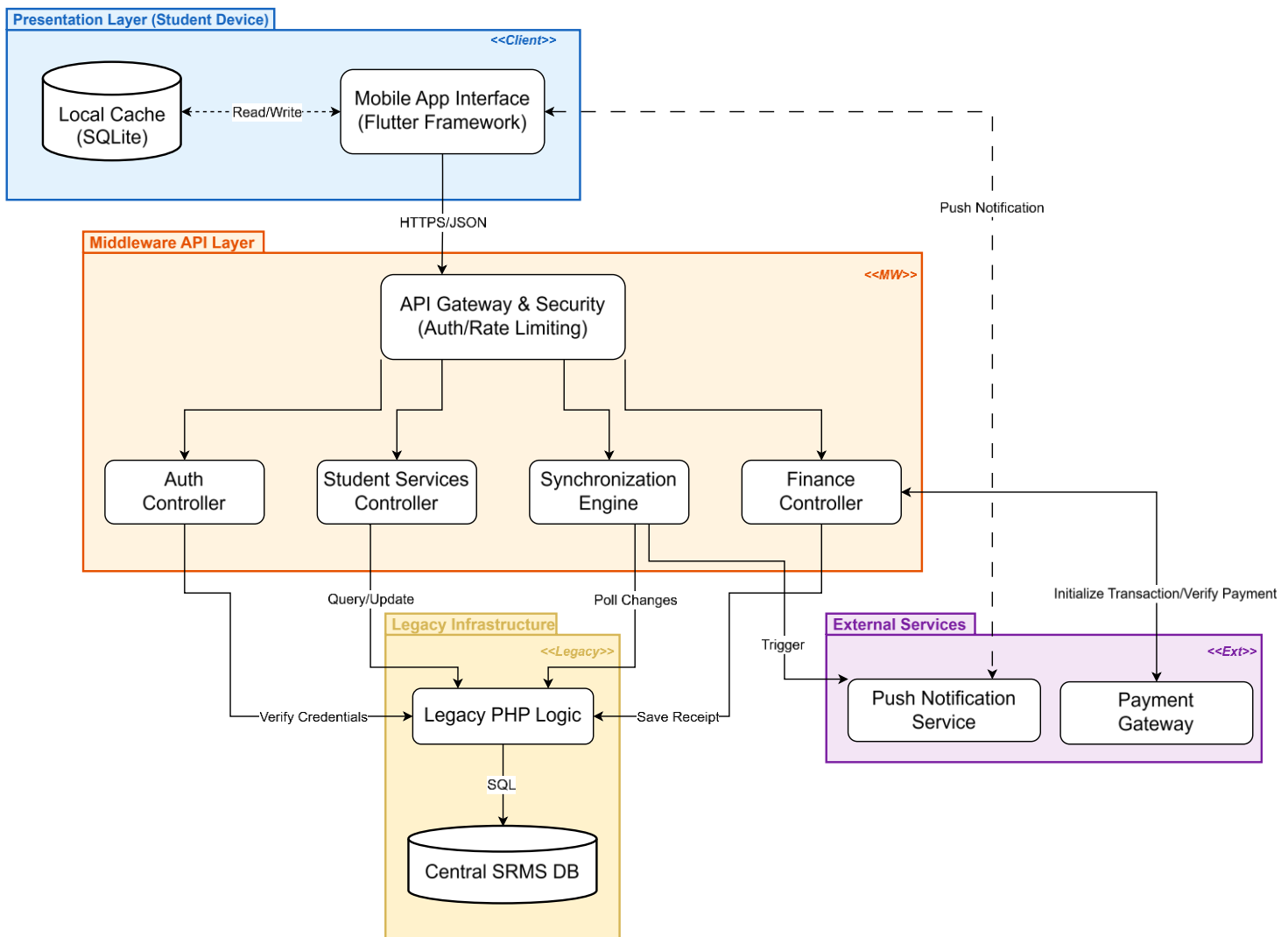
The activity diagram above shows the entire workflow of how different types of students interact with the SRMS mobile app and how the system process each action in its backend. The diagram is organized using 6 vertical swimlanes (prospective student, local/foreign student, foreign student only, SRMS mobile app, middleware API & legacy DB and the payment gateway) all in different colours for clear visuals and to separate which actor or system component is responsible for each activity. The flow starts in the mobile app lane with the student opening the app followed by a decision node which tells us whether the user is a prospective or current (local/foreign) student. The prospective student lane (green) consists of the application process activities such as entering personal information, selecting a program, uploading the required documents and reviewing the submission. Each of these steps is then validated through the middleware API & legacy DB lane which stores the data and returns success or error messages. After the application is submitted, the mobile app sends an application submitted notification to the user. For local and foreign students (blue lane), the diagram shows the course registration workflow. Student begins course registration, review available courses, add or drop courses and then submit their registration. These steps depend on backend actions like getting the program info, validating prerequisites and saving the final registration when they finish. Once it is submitted successfully, the app sends the student a registration notification. The foreign student only lane (light pink) shows the payment steps that only happen for students who are not Guyanese. This includes viewing invoices, selecting “pay now” and then the app sending them to the payment screen. The payment gateway lane is the one that actually handles the payment and send back either a success message so the backend can update the student balance and status or it sends an error telling the student to try again. After that the diagram shows some academic features that both local and foreign students will use like opening the timetable and viewing grades. Local students will get access to this after they successfully register while foreign students will get access after they go through the payment process. When the student open the timetable, the app asks the backend for their courses and the timetable details and the system sends back the final timetable. For grades, the app requests the grades and GPA from the database, the system calculates the GPA and then it shows the student the results. The app also send a notification whenever a new grade gets posted. Each flow in the diagram ends with a final node to show that part of the process is finished.

7.0 System Architectural Design

The SRMS mobile application is designed using a Layered Client-Server Architecture. This approach was selected specifically because the system must work with the University's existing system. Since the PHP system cannot be altered, a layered design lets us build a modern mobile app while keeping the business logic and data storage separate.

This structure ensures that the mobile app (Client) never talks directly with the central database. Instead, it connects through a secure Middleware layer. This separation maintains security and if the University updates the PHP code in the future, it will not immediately crash the mobile application.

7.1 System Architectural Diagram Figure



7.2 System Architectural Diagram Description

The architecture is divided into four main layers:

1. Presentation Layer (Student Device)

This layer represents the client side of the system. It contains the Mobile App Interface, which is what the student interacts with, developed using the Flutter Framework to ensure the app runs smoothly on the older Android devices used by many students. This layer also includes a Local Cache (SQLite) which allows the app to store data on the student's device to view information even if their internet connection is unstable.

2. Middleware API Layer

This layer connects the mobile app and the university's existing systems.

- **API Gateway & Security:** This is where all requests coming from the mobile app are processed. It handles security tasks like authentication and rate limiting to protect the system.
- **Controllers:** The diagram shows specific controllers (Authorization, Student Services, Finance) that handle the logic for different features.
 - **Auth Controller:** Handles logins and sends “Verify Credentials” requests to the legacy system.
 - **Student Services Controller:** Handles the academic section like registration, grades and timetables.
 - **Finance Controller:** Handles payments by starting the “Initialize Transaction” request and waiting for the processing of payment confirmations.
- **Synchronization Engine:** This component manages the data flow between the legacy database and the mobile app to ensure the student sees up to date information.

3. Legacy Infrastructure

This layer represents the university's existing systems.

- **Legacy PHP Logic:** This is the current backend logic used by the university. The new system interacts with this rather than rewriting everything.
- **Central SRMS DB:** This is the main database where all student records are currently stored. It serves as the primary data repository.

4. External Services

These are third-party systems used for specific tasks.

- **Push Notification Service:** Used to send alerts to the student's device.
- **Payment Gateway:** Used to process fees securely.

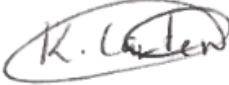
7.3 Scenario: Course Registration

This scenario explains how the components work together when a student registers for a new course, such as “CSE2101”.

- **Student Action:** The student selects the course in the Mobile App Interface (Presentation Layer) and clicks “Register.”
- **Request:** The app sends the registration data through the API Gateway in the Middleware Layer.
- **Routing:** The Gateway recognizes this as a student action and passes the request to the Student Services Controller.
- **Validation:** The controller connects to the PHP Logic to check if the student has the required prerequisites and if the class has space available.
- **Database Update:** If the validation passes, the Legacy Logic runs an SQL command to update the Central SRMS DB. If the student meets the requirements, it updates the database to confirm the registration.
- **Final Confirmation:** The legacy system confirms the success of the middleware, which sends a message back to the mobile app. The app then updates the Local Cache (SQLite) so the student can immediately see the course on their timetable.

Collaborative Summary

Division of Labour:

Name	Tasks	Signature
Kaylee Carter	Sequence Diagrams (Simple & Complex)	
Khaleel Khan	Context Model & Description	
Ravindra Persaud	Detailed Class Diagram	<i>R. Persaud</i>
Vimla Persaud	Use Case Model & Activity Diagram	<i>Vimla Persaud</i>
Tulsidai Singh	System Architectural Design	<i>T. Singh</i>

Collaboration Strategies and General Experiences:

Our group used a shared Google Doc for real-time collaborative writing and a WhatsApp group for daily communication and coordination. This strategy worked well, as the shared document allowed everyone to see progress and contribute easily. Regular check-in meetings were held via Google Meet to ensure everyone's Diagrams aligned.

Challenges and Resolutions:

Our main challenge was ensuring a consistent diagram style across the document, since each diagram was done by a different person. We handled this by using one website for the diagrams.

Appendix A: High-Resolution Diagram Links

Note: This appendix is included to ensure diagram clarity, as per best practice for large submissions. All diagrams are embedded in the main document.

https://drive.google.com/drive/folders/13iNI-DovUIk4K7TmnVewzkIS5g_v8Khm